EDGE-SECURE LAB

ANSIBLE & SYSTEMD RESILIENCE

(Phase 3 — Ansible & Systemd Resilience)

BY: UDOFIA, EDINEN SUNDAY
(Systems Administrator & I.T Support Engineer)

5th MAY, 2026

1. PHASE OVERVIEW

This phase focused on transforming the existing hybrid infrastructure into an automated and resilient environment using:

- Ansible for infrastructure orchestration
- SSH trust relationships for remote execution
- systemd for service lifecycle management
- YAML-based Infrastructure as Code (IaC)
- Linux service resilience engineering
- Bastion-based administration
- Enterprise-style troubleshooting methodology

This phase represented a major transition from:

"manually managing systems"

to:

"engineering reproducible infrastructure"

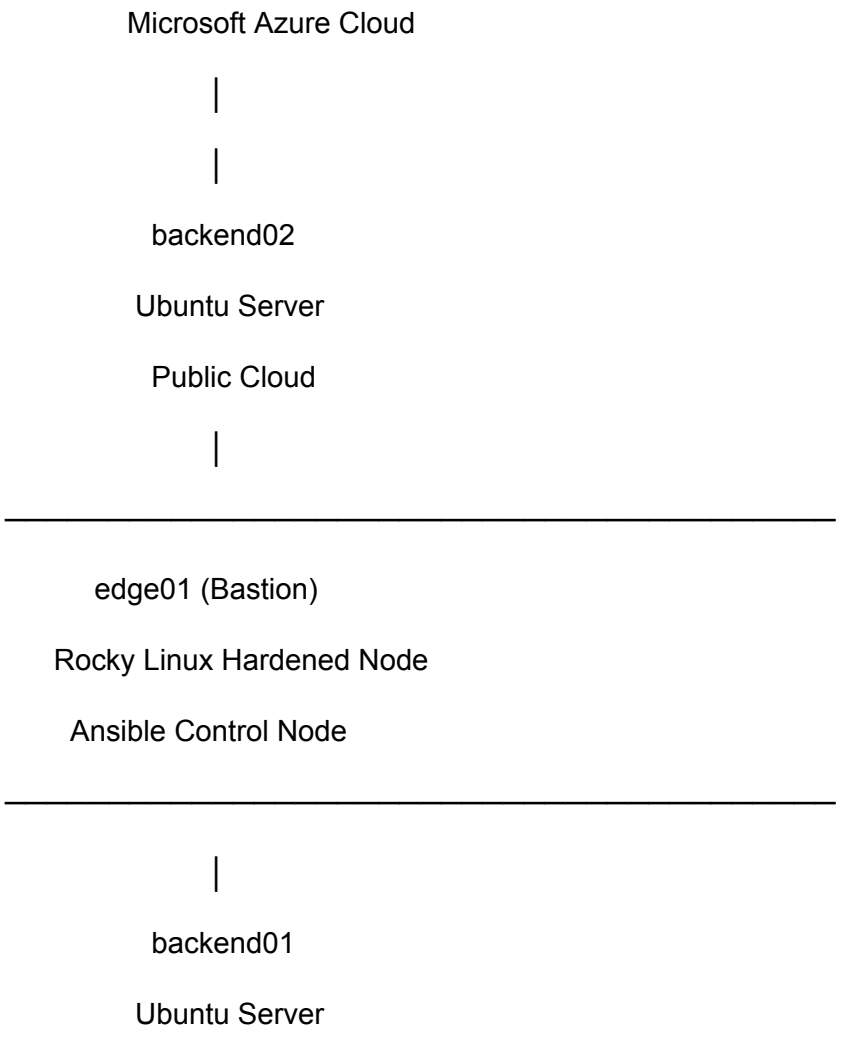
The environment now supports:

- centralized orchestration
- repeatable deployments
- automated configuration management
- resilient services
- hybrid infrastructure management
- controlled SSH-based administration

---

2. FINAL LAB ARCHITECTURE

2.1 Infrastructure Topology



Local VM Backend

---

3. SYSTEM ROLES

3.1 edge01

Platform:

- Rocky Linux

Purpose:

- Bastion host
- Hardened edge node
- Ansible control node
- SSH jump server
- Central orchestration system

Key Services:

- firewalld
- SSH
- Ansible
- systemd

3.2 backend01

Platform:

- Ubuntu Server

Purpose:

- Local managed node
- Apache deployment target
- Internal infrastructure node

Network:

- Host-only network

3.3 backend02

Platform:

- Ubuntu Server
- Azure VM

Purpose:

- Cloud managed node
- Apache deployment target
- Hybrid cloud integration node

Network:

- Public cloud networking
- Azure NSG controlled access

---

4. CORE ENGINEERING CONCEPTS

4.1 WHAT IS ANSIBLE?

Ansible is an infrastructure automation engine used for:

- configuration management
- orchestration
- software deployment
- infrastructure provisioning
- compliance enforcement

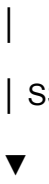
Ansible communicates over SSH.

No permanent software agent runs on managed nodes.

This architecture is called:

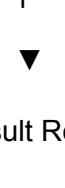
4.2 ANSIBLE OPERATIONAL FLOW

Ansible Controller (edge01)



Remote Node

Temporary Python Module Executed



Result Returned to Controller

4.3 WHAT IS IDEMPOTENCY?

Idempotency means:

Running the same automation repeatedly

produces the same final state.

Example:

If Apache is already installed:

state: present

Ansible does NOT reinstall it.

Instead:

changed=0

This prevents:

- duplicate actions
- unnecessary downtime
- unstable automation
- inconsistent infrastructure states

4.4 WHAT IS SYSTEMD?

systemd is the Linux init system.

It is:

- PID 1
- the first userspace process started by the kernel
- the process manager controlling services

Verification:

ps -p 1

Expected Output:

systemd

4.5 WHAT IS A SERVICE?

A service is:

A background process managed by systemd.

Examples:

- sshd
- apache2
- rsyslog
- haproxy

systemd manages:

- startup
- shutdown
- restarts
- failures
- boot persistence

4.6 WHAT IS RESILIENCE?

Resilience means:

A system can recover automatically from failure.

In this lab:

- a service intentionally crashes
- systemd detects failure
- systemd automatically restarts the service

This simulates:

- enterprise recovery behavior
- self-healing infrastructure

---

5. SSH ARCHITECTURE & ACCESS MODEL

5.1 SSH DESIGN USED IN THIS LAB

SSH access hierarchy:

Windows Host

Operationally:

- Windows connects to edge01
- edge01 acts as bastion host
- edge01 manages all infrastructure

This mirrors real enterprise administration models.

5.2 WINDOWS SSH CONFIGURATION

A Windows SSH config file was created to simplify access.

Purpose:

Instead of typing:

ssh -i key.pem eudofia@IP

You can simply type:

ssh edge01

5.3 WINDOWS SSH CONFIG FILE LOCATION

Windows OpenSSH config location:

C:\Users\<USERNAME>\.ssh\config

5.4 WINDOWS SSH CONFIG CONTENT

Example:

Host edge01

HostName <EDGE01_IP>

User eudofia

IdentityFile C:\Users\<USERNAME>\.ssh\id_ed25519

5.5 WHY THIS MATTERS

This improves:

- operational speed
- administrative consistency
- repeatability
- workflow efficiency

This is common in:

- DevOps
- SRE
- Infrastructure Engineering
- Cloud Operations

5.6 TMUX USAGE

After SSH into edge01:

tmux

Purpose:

- persistent sessions
- multitasking
- survive terminal disconnects
- enterprise operational workflow

This prevents:

- losing sessions during disconnects
- losing long-running tasks

---

6. INSTALLING ANSIBLE

6.1 INSTALL COMMAND

Executed on edge01:

sudo dnf install -y ansible-core

6.2 COMMAND BREAKDOWN

Component	Meaning
dnf	Rocky Linux package manager
install	install package
-y	auto-confirm prompts
ansible-core	minimal Ansible engine

6.3 WHY ANSIBLE-CORE?

ansible-core contains:

- Ansible execution engine
- modules
- playbook runner
- SSH orchestration engine

It excludes:

- large optional collections
- unnecessary packages

This keeps systems lightweight.

---

7. BUILDING THE INFRASTRUCTURE DIRECTORY

7.1 CREATE WORKSPACE

mkdir ~/infrastructure

cd ~/infrastructure

Purpose:

Centralized storage for:

- inventories
- playbooks
- configuration files
- automation assets

---

8. INVENTORY ENGINEERING

8.1 WHAT IS AN INVENTORY?

An inventory defines:

- managed systems
- connection details
- authentication settings
- grouping logic

Without inventory:

Ansible does not know:

- where to connect

- which user to use
- which hosts belong to which environment

8.2 FINAL INVENTORY FILE

File:

vim inventory.ini

Content:

[cloud]

backend02 ansible_host=158.158.37.102 ansible_user=ubuntu
ansible_ssh_private_key_file=/home/eudofia/.ssh/id_ed25519

[local]

backend01 ansible_host=192.168.56.11 ansible_user=eudofia

8.3 INVENTORY FIELD BREAKDOWN

Group Definitions

[cloud]

Logical group name.

Used for:

- targeting systems
- environment segmentation
- scalable automation

Host Alias

backend02

Human-readable hostname alias.

Instead of using raw IPs everywhere.

---

ansible_host

ansible_host=158.158.37.102

Real IP address Ansible connects to.

---

ansible_user

ansible_user=ubuntu

SSH username used for login.

---

ansible_ssh_private_key_file

ansible_ssh_private_key_file=/home/eudofia/.ssh/id_ed25519

Private key used for authentication.

---

9. INVENTORY TROUBLESHOOTING

9.1 INITIAL FAILURE

Observed Error:

Could not match supplied host pattern

and:

Parsed /etc/ansible/hosts inventory source

9.2 ROOT CAUSE

Ansible was using:

/etc/ansible/hosts

instead of:

inventory.ini

Therefore:

- backend02 did not exist in active inventory
- host resolution failed
- automation failed before SSH authentication even began

9.3 FIX

Correct execution:

ansible -i inventory.ini backend02 -m ping

and:

ansible-playbook -i inventory.ini deploy.yml

9.4 PERMANENT FIX USING ansible.cfg

Created:

vim ansible.cfg

Content:

[defaults]

inventory = inventory.ini

host_key_checking = False

9.5 WHY THIS FIX MATTERS

This ensures:

- consistent inventory loading
- reduced command repetition
- simplified automation workflow

---

10. SSH AUTHENTICATION TROUBLESHOOTING

10.1 INITIAL ERROR

Observed:

Permission denied (publickey)

10.2 IMPORTANT DISCOVERY

Manual SSH worked:

ssh ubuntu@158.158.37.102

But Ansible failed.

This proved:

- Azure VM reachable
- NSG rules functional
- SSH daemon functional
- networking operational

The problem was not network connectivity.

10.3 ROOT CAUSE

SSH private key:

~/.ssh/id_ed25519

was encrypted with a passphrase.

Manual SSH worked because:

- interactive terminal allowed passphrase entry

Ansible initially failed because:

- automation sessions require ssh-agent support

10.4 SSH AGENT FIX

Start SSH agent:

eval "$(ssh-agent -s)"

Add key:

ssh-add ~/.ssh/id_ed25519

Verify:

ssh-add -l

10.5 WHY SSH-AGENT MATTERS

ssh-agent:

- securely stores decrypted private keys in memory
- avoids repeated passphrase prompts
- enables automation tools to authenticate non-interactively

This is heavily used in:

- CI/CD systems
- DevOps pipelines
- enterprise automation

11. FIREWALL & CONNECTIVITY TROUBLESHOOTING

11.1 OBSERVED NETWORK BEHAVIOR

Source	Destination	Result
edge01 → backend01	Ping	Success
edge01 → backend02	Ping	Success
backend01 → edge01	Ping	Failed
backend02 → edge01	Ping	Failed
edge01 → backend02	SSH	Success

11.2 INITIAL CONCERN

It initially appeared:

- the network might be broken
- Azure might be blocking ICMP

- routing might be failing

However:

This was actually expected firewall behavior.

11.3 ROOT CAUSE

edge01 had:

- hardened firewall policy
- default DROP inbound behavior
- explicit SSH allow rules

This meant:

- outbound initiated traffic succeeded
- established/related traffic returned successfully
- unsolicited inbound ICMP was dropped

11.4 PACKET-LEVEL ANALYSIS

Successful Ping From edge01

edge01 → ICMP Echo Request → backend02

backend02 → ICMP Echo Reply → edge01

Reply traffic allowed because:

RELATED/ESTABLISHED

state existed.

Failed Ping From backend02

backend02 → ICMP Echo Request → edge01

Dropped because:

- INPUT policy = DROP
- no explicit ICMP allow rule existed

11.5 ENGINEERING CONCLUSION

The network was functioning correctly.

The firewall was successfully enforcing:

Zero Trust Principles

11.6 FIREWALL VERIFICATION COMMANDS

firewalld

sudo firewall-cmd --list-all

sudo firewall-cmd --list-rich-rules

---

nftables

sudo nft list ruleset

---

iptables

sudo iptables -L -n -v

11.7 OPTIONAL INTERNAL ICMP RULE

sudo firewall-cmd \

--permanent \

--add-rich-rule='rule family="ipv4" \

source address="192.168.56.0/24" \

protocol value="icmp" accept'

Reload:

sudo firewall-cmd --reload

---

12. AZURE NSG INVESTIGATION

12.1 ACTION TAKEN

Azure NSG inbound ICMP rule was added.

12.2 RESULT

Ping still failed.

12.3 CONCLUSION

Azure was not the problem.

edge01 firewall hardening caused the behavior.

This was an important engineering lesson:

Cloud firewall troubleshooting

must consider BOTH:

- cloud-layer controls

- host-layer controls

---

13. THE PLAYBOOK

13.1 PLAYBOOK CREATION

File:

vim deploy.yml

13.2 FINAL WORKING PLAYBOOK

```
---

- name: Hybrid Apache Deployment

  hosts: all
```

```
become: yes

tasks:
  - name: Install Apache on Debian systems
    apt:
      name: apache2
      state: present
      update_cache: yes
    when: ansible_os_family == "Debian"

  - name: Ensure Apache is running
    systemd:
      name: apache2
      state: started
      enabled: yes
    when: ansible_os_family == "Debian"

  - name: Deploy index page
    copy:
      content: "<h1>{{ inventory_hostname }}</h1>"
      dest: /var/www/html/index.html
    when: ansible_os_family == "Debian"

  - name: Configure UFW firewall
```

```
community.general.ufw:
  rule: allow
  name: Apache Full
when: ansible_os_family == "Debian"
```

13.3 PLAYBOOK BREAKDOWN

hosts: all

Target:

Every host in inventory.

---

become: yes

Purpose:

Privilege escalation using sudo.

Required because:

- package installation requires root
- firewall changes require root
- service management requires root

13.4 TASK ANALYSIS

TASK 1 — INSTALL APACHE

```
apt:
  name: apache2
  state: present
  update_cache: yes
```

Meaning

Field	Meaning
name	package name
state: present	ensure installed
update_cache	run apt update

TASK 2 — START APACHE

```
systemd:
  name: apache2
  state: started
  enabled: yes
```

Meaning

Field	Meaning
state: started	ensure service running
enabled: yes	start on boot

TASK 3 — DEPLOY INDEX PAGE

content: "<h1>{{ inventory_hostname }}</h1>"

Jinja2 variable substitution.

Dynamically inserts:

- backend01
- backend02

This proves:

- automation executed correctly
- host-specific deployment occurred

TASK 4 — CONFIGURE UFW

community.general.ufw:

Allows:

Apache Full

Meaning:

- port 80 allowed
- port 443 allowed

---

14. YAML TROUBLESHOOTING

14.1 ISSUE

Playbook initially failed.

Cause:

Incorrect indentation.

14.2 WHY YAML IS SENSITIVE

YAML uses:

Whitespace as syntax.

Improper indentation changes:

- structure
- nesting
- task hierarchy

14.3 LESSON LEARNED

Infrastructure as Code requires:

- syntax discipline
- structural consistency
- whitespace accuracy

15. ANSIBLE COLLECTION ISSUE

15.1 ERROR

couldn't resolve module/action 'community.general.ufw'

15.2 ROOT CAUSE

Required collection missing.

15.3 FIX

Installed collection:

ansible-galaxy collection install community.general

15.4 WARNING OBSERVED

Collection community.general does not support Ansible version 2.14.18

15.5 ENGINEERING INTERPRETATION

This was:

Compatibility warning

NOT:

Fatal execution failure

The module still functioned.

---

16. SUDO PRIVILEGE ESCALATION ISSUE

16.1 OBSERVED PROBLEM

Ansible required privilege escalation.

Managed nodes needed passwordless sudo.

16.2 FIX IMPLEMENTED

On backend systems:

su -

Then:

echo "eudofia ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/eudofia

---

16.3 WHY THIS WAS REQUIRED

Ansible executes tasks remotely.

Without passwordless sudo:

- automation pauses
- sudo password prompts appear
- orchestration becomes unreliable

16.4 ENTERPRISE IMPLICATION

This demonstrates:

- privilege escalation management
- automation trust boundaries
- operational delegation

---

17. PLAYBOOK EXECUTION WORKFLOW

17.1 SYNTAX CHECK

ansible-playbook -i inventory.ini deploy.yml --syntax-check

Purpose:

Validate YAML structure before deployment.

17.2 DRY RUN

ansible-playbook -i inventory.ini deploy.yml --check

Purpose:

Simulation mode.

Shows:

- intended changes
- without modifying systems

17.3 VERBOSE MODE

ansible-playbook -i inventory.ini deploy.yml -vvv

Purpose:

Deep troubleshooting visibility.

Displays:

- SSH negotiation
- module execution
- connection flow
- task details

17.4 FINAL EXECUTION

ansible-playbook -i inventory.ini deploy.yml

17.5 SUCCESS INDICATORS

Expected:

ok=...

changed=...

failed=0

---

18. WEB VERIFICATION

18.1 TEST LOCAL BACKEND

curl 192.168.56.11

Expected:

<h1>backend01</h1>

18.2 TEST CLOUD BACKEND

curl <cloud-ip>

Expected:

<h1>backend02</h1>

---

19. SYSTEMD RESILIENCE ENGINEERING

19.1 PURPOSE

A deliberately unstable service was engineered to:

- simulate crashes
- observe recovery behavior
- understand systemd restart policies

19.2 CREATE SCRIPT

sudo vim /usr/local/bin/test_service.sh

Content:

#!/bin/bash

echo "Service running..."

sleep 5

exit 1

19.3 SCRIPT LOGIC

Line	Meaning
#!/bin/bash	interpreter

echo	output message
sleep 5	wait 5 seconds
exit 1	intentional failure

19.4 MAKE EXECUTABLE

sudo chmod +x /usr/local/bin/test_service.sh

19.5 CREATE SYSTEMD UNIT

sudo vim /etc/systemd/system/test.service

Content:

[Unit]

Description=Test Resilience Service

After=network.target

[Service]

ExecStart=/usr/local/bin/test_service.sh

Restart=on-failure

RestartSec=3

[Install]

WantedBy=multi-user.target

19.6 UNIT FILE BREAKDOWN

[Unit]

Description=Test Resilience Service

Human-readable identifier.

After=network.target

Wait until networking initializes.

[Service]

ExecStart=

Command executed.

Restart=on-failure

Restart if:

- non-zero exit code
- crash
- abnormal termination

RestartSec=3

Wait 3 seconds before restart.

[Install]

WantedBy=multi-user.target

Enable service during normal multi-user boot.

19.7 ACTIVATE SERVICE

sudo systemctl daemon-reexec

sudo systemctl daemon-reload

sudo systemctl enable --now test.service

19.8 COMMAND BREAKDOWN

Command	Meaning

daemon-reexec	restart systemd process
daemon-reload	reload unit files
enable	start at boot
--now	start immediately

19.9 VERIFY SERVICE

systemctl status test.service

Expected:

activating (auto-restart)

or repeated restart cycles.

---

20. FAILURE SIMULATION

20.1 BREAK THE SERVICE

sudo chmod -x /usr/local/bin/test_service.sh

20.2 RESULT

systemd attempted execution.

Kernel denied execution.

Service failed immediately.

20.3 FIX

sudo chmod +x /usr/local/bin/test_service.sh

sudo systemctl restart test.service

---

21. ENTERPRISE ENGINEERING LESSONS

21.1 CONNECTIVITY LAYERS ARE INDEPENDENT

Important realization:

ICMP failure ≠ SSH failure

SSH success ≠ Ansible success

Different layers fail independently.

21.2 HARDENED SYSTEMS BEHAVE DIFFERENTLY

Enterprise systems:

- deny unsolicited traffic
- restrict inbound communication
- permit only explicit services

This is expected behavior.

21.3 INVENTORY MANAGEMENT IS CRITICAL

Automation reliability depends on:

- correct inventories
- accurate host mapping
- proper group definitions
- identity consistency

21.4 SSH AUTOMATION REQUIRES IDENTITY MANAGEMENT

Automation systems must properly manage:

- SSH keys
- agents
- passphrases
- authentication flow

21.5 BASTION ARCHITECTURE PRINCIPLES

edge01 now behaves like:

- enterprise jump server
- hardened administrative boundary
- orchestration node
- trust control point

21.6 INFRASTRUCTURE AS CODE DISCIPLINE

IaC requires:

- syntax accuracy
- consistency
- repeatability
- verification workflows

---

22. FINAL VALIDATION CHECKLIST

Validation	Status
SSH from Windows → edge01	Success
SSH from edge01 → backend01	Success
SSH from edge01 → backend02	Success
Inventory recognized	Success
Ansible playbook executed	Success
Apache installed	Success

Apache enabled	Success
Custom HTML deployed	Success
UFW configured	Success
systemd resilience service functioning	Success
Automatic restart observed	Success

---

23. OPERATIONAL COMMAND REFERENCE

23.1 SSH Agent

```
eval "$(ssh-agent -s)"

ssh-add ~/.ssh/id_ed25519

ssh-add -l
```

23.2 Ansible

```
ansible -i inventory.ini all -m ping
```

ansible-playbook -i inventory.ini deploy.yml

ansible-playbook -i inventory.ini deploy.yml --check

ansible-playbook -i inventory.ini deploy.yml --syntax-check

ansible-playbook -i inventory.ini deploy.yml -vvv

23.3 systemd

systemctl status test.service

systemctl restart test.service

systemctl daemon-reload

23.4 Firewall

firewall-cmd --list-all

nft list ruleset

iptables -L -n -v

---

24. FINAL ENGINEERING STATE

At the end of this phase, the infrastructure now supports:

24.1 Infrastructure Automation

- centralized orchestration
- YAML-driven deployments

- repeatable provisioning

24.2 Hybrid Infrastructure Management

- local VM administration
- cloud VM administration
- bastion-based operations

24.3 Secure Administration

- SSH key authentication
- hardened firewall posture
- privilege escalation control

24.4 Service Resilience

- automatic service recovery
- restart policies
- controlled failure handling

24.5 Enterprise Troubleshooting Methodology

Practiced skills:

- root cause analysis
- layered troubleshooting
- inventory debugging
- firewall analysis
- SSH diagnostics
- automation debugging
- YAML troubleshooting
- cloud networking analysis